Practical 3. Getting Started with Git and GitHub

Rob Young, Caitlin McNiff, Zhengyang Yuan

IBI1, 2025/26

1 Learning objectives

- Explain the principles of version control
- Manage projects with git and github
- Build a simple website using github pages

2 Git

- Here, you will start a small version control project and learn to work with Git.
- Open Visual Studio Code and **use the Terminal** to navigate to the folder on your computer where you save your IBI1 notes. Create a directory IBI1_2025-26 within this directory and move into that directory **using the Terminal**.
- Start a local project by typing `git init`. You should see a message similar to the following on your terminal window: `Initialized empty Git repository in /Users/<USERNAME>/IBI1/IBI1_2025-26/.git/`
- Create a new text file called 'README.md'. Edit this file to include a few lines describing the project. For instance: "This is my first git project." Once you have added the text, save the file.
- Add this file to your repo by typing `git add README.md` in the Terminal window. You have added a file to your repository but you haven't yet committed the changes. Commit the changes with the command `git commit -m "Initialise"`. The -m option adds the commit message so you can set it to a different message if you like. You should see an output message stating that you have 1 file changed with 1 insertion.
- Look in your log file by typing `git log --oneline`. Can you see your initial commit?
- Now it's time to create a new file. Create a new text file, **thingsIlike.txt**. Use this file to make a list of 10 things you like, in this format:

  I like tea
  I like chocolate biscuits
  I like snow…

- Add this file to your repository and commit these changes. What does your log look like now?
- Now, change your mind! Add one extra thing you like. Commit by typing `git commit -m "edit thingsIlike"`. Does this error message make sense? Now add the modified file and try committing again.
- Now, change your mind again! Commit back to the previous version (the original list of 10 things). You can do this by typing `git revert HEAD`. This command displays a commit

message which you can escape by typing `:x` (full disclosure: this message is displayed in a text editor vim, similar to nano). What happens next? How does your log change?

- Create a new branch with a friend's name (e.g. Alex) by typing `git branch Alex` and switch to the friends branch by typing `git checkout Alex.`
- In the friend branch, remove 3 things you don't like. Add 5 things you like. Change one "like" to "don't like". Add and commit these changes as before.
- Move back to the master branch (`git checkout master`). Make a few changes to your favourite things yourself: Remove one thing, add one thing, change one "like" to "like very much"
- Merge the friend branch into your master branch. First move back to the master branch if you have not already done so and then type `git merge Alex` (assuming that the new branch you made was called Alex). You may need to escape the commit message as previously for `git revert`. What does thingsIlike.txt look like now? Did you have any conflicts?

3 Getting started with GitHub

- Here, you will start a small version control project and learn to work with Git
- Go to GitHub: https://github.com/
- Sign up. You will be asked to choose a username, and to provide an e-mail address and a password. Your username can be anything (as long as it is not already taken), but bear in mind that you will probably have your GitHub account for a long time, so choose a name that is sensible. (Bear in mind also that your instructors will see your GitHub repo!).
- You will be asked to solve a "Captcha" puzzle to prove that you are a human, not a bot. Once you have done this, choose your plan. We will be using the "Free" version in all of our courses, so please select this. Click "Continue". There is a little questionnaire (which you can skip). You will then be asked to verify your e-mail address. Check your e-mail (including your spam folder) for a message from GitHub, and follow the instructions. Congratulations, you now have a GitHub account.
- At the moment, you don't have any repository in GitHub. In this practical, you will make two: One by forking an existing repository, and one by creating your own (see below). We will start with the fork. Go to the following link: https://github.com/r0bah0lic/IBI1_2025-26. You will be shown the **IBI1_2025-26** repository. You can have a look at the file structure etc. Click "Fork" (top right) to create a fork of the repository. This may take a bit of time. But when it's done you will have your own repository called **IBI1_2025-26**
- You have successfully created a repository in GitHub, but to really be able to work with it, you want a copy of it on your own computer. You can do this by typing the following command: `git clone https://github.com/USERNAME/IBI1_2025-26`. Make sure to change USERNAME to your own username that you just selected. You may have to wait a few seconds for everything to download but afterwards you should see an `IBI1_2025-26` directory. **Use the Terminal** to explore the file structure, is it the same as you saw on GitHub?
- Move into this directory and have a look at the repo. What information about previous commits can you see?

- Now is your time to make this directory your own. Find the file **Practical3/Introductions.txt**. Edit it and replace the information about Rob with information about yourself. Commit your edits.
- Copy the file **thingsIlike.txt** you made in Part 1 into this repository (into the directory called **Practical3**). Commit your changes. (This is not something you would normally do very often, but in this case it makes sense).

- Now, how to move your local changes back to GitHub? Type the following command `git push origin master`. Make sure you use your username and password to move these changes back to your own GitHub account! If you get permission denied errors, you should try this command `git config –local credential.helper “”` before trying `git push origin master` again.

- Go back to your browser and look at your GitHub again. You should see that the content of your repository has changed. In directory **Practical3**, you should see that the file **Introductions.txt** has recently changed, and you should see your own commit message. You should also see the additional file **thingsIlike.txt**. Compare this to the original repo you forked from: https://github.com/r0bah0lic/IBI1_2025-26. This has not changed. (That's the way it should be. Your fork is for you to edit and play around with, but it will not change the "Master" copy of practical instructions).

- Add a file called **Reflection.txt** to the directory **Practical3**. In this file, briefly note your thoughts about the Practical. Did everything go well? Are there things that were challenging? What questions do you have?

4 Making a website with GitHub pages

GitHub can also be used to make a simple website. We will do this here.

- In GitHub, create a new repository by clicking on the "+" symbol in the upper right corner, and then selecting "New repository". Your repository needs to have a very specific name: ***username*.github.io**, where ***username*** is your github username. When you create your repository, you can add a description if you want. Make the repository public. Do not click "Initialize this repository with a README".

- Move back to your home directory and clone your ***username*.github.io** repo.

- There is a file called **index.md** in **IBI1_2025-26/Practical3**. Copy that to your ***username*.github.io** repo. Replace the old file. Commit. Push the changes to GitHub. In your web browser, type ***username*.github.io**. You can also find the link in "GitHub Pages" section. What do you see? Look at the **index.md** file in your text editor or in GitHub. Do you understand how it works? Can you edit it, so your web page shows information about yourself? Here is an example we did: https://r0bah0lic.github.io

- Now it is time to choose a style for your website. Here, we will use a pre-defined style through a framework called Jekyll. Create a new filled called **_config.yml** inside your ***username*.github.io** repo – this stores the information necessary to achieve a design.

- In your ***username*.github.io** repository, go to "Settings". It's OK to leave the options as they are. Go to the "Pages" section and click on "add a Jekyll theme to your site". Click on "Supported themes". Pick one you like and follow the instructions to add this theme to your **_config.yml** theme.

5 For your portfolio

The markers will look for and assess the following:

- A simple website about yourself at ***username*.github.io** with some kind of styling (no plain text), and a little bit of text that is different from the index.md file you were given. You can use this as your personal webpage and add whatever information you like. It does not need to stay the same as it is at the end of this practical.
- In your fork of **IBI1_2025-26/Practical3**, you should have a file called **Introductions.txt** that has been edited by you and that contains information about yourself.
- In your fork of **IBI1_2025-26/Practical3**, you should have another file called **thingsIlike.txt** that contains a list of things you like.
- Your fork of **IBI1_2025-26/Practical3** should contain a file called **Reflection.txt** which contains some thoughts on this practical.
- The commit messages on your fork of **IBI1_2025-26/Practical3** should be relatively informative. (They don't have to be perfect, but they shouldn't all be empty, and they should have information about the kind of change that was made, not just "Commit 4" or "blablabla")

You can add or edit things after the Practical session. We do not look at the commit date, we just want it all to be there!